

SOD226C — Practical Assessment 2 · 2026

Marking rubric

Read this before you start.

Central University of Technology, Free State — Diploma in IT Software Development 2B
(SOD226C) · Lecturer: Mr P Kruger

Released **Monday 21 September 2026**. Due **Friday 2 October 2026, 23:00**.

How this assessment is marked

Marked out of **50** and reported to you as a **percentage**. Each mark is worth 2 %.

One submission per group, in a folder named with the **Group Leader's student number**. Every member of the group receives the same mark.

Your work is marked both by **running the application** and by **reading the code**. An application that behaves correctly but was not built as the brief requires does not earn full marks, and neither does code that reads correctly but does not run.

This assessment is not Practical Assessment 1 with a database bolted on. It is a new project, and the marks have moved. In PA1 most of the marks were for making the application work. Here a working application earns you Block A, and Block B asks whether you built it the way Unit 4 teaches. The two blocks are close to the same size, and Block B is the harder one.

There are no worksheets for this assessment. Your sources are the three tutorial documents named in §1.1 of the brief. They build the Student Management App; you are building the Board App. Working out what changes between the two is part of what is being assessed.

Two things that end the marking before it starts

The folder name is wrong	The project folder must carry the Group Leader's student number and nothing else. If it does not, your assignment is not graded .
The solution does not build	0 for the assessment. This is the same rule as SOD216C and as Practical Assessment 1. A program that does not compile has not been written.

The second one is not a threat, it is arithmetic. Building your solution takes seconds, and there is no version of this assessment where you should be submitting something you have not built.

Where the marks are

Block	What is assessed	Marks
A	Behaviour: the application, run, used, and then stopped and started again	20
B	Implementation: the code, read against the requirements brief	25
C	Comment Coding Standard	5
	Total	50

Block A — behaviour · 20

Assessed	Marks
The database is created in the right place, with the right table, and the ten seed boards appear on first run	3
Boards can be listed, and one board's details viewed	2
A valid board is added and confirmed; an invalid board is refused with the right messages; a board code that already exists is refused rather than crashing	4
An existing board is changed, the change lasts, and its board code is protected	3
A board is deleted and the deletion confirmed	2
The application is stopped and started again, and everything you did is still there, the board you added, the change you saved, the board you deleted, and the ten seed boards have not duplicated	4
The home page and navigation are as the brief requires	2

Block B — implementation · 25

Assessed	Marks
The database context is correct. The DbSet and table are named as specified, and Price is configured so that it sorts correctly	3
Dependency injection is wired up in Program.cs, and the database is initialised at start-up in the way the framework requires	5
The initialiser receives its context rather than creating one, creates the database, and seeds it only when the table is empty	3
The board repository implements every member of the interface correctly against the database, re-using its own code where the standard requires it	5
The controller receives its repository rather than creating one, has the required signatures, and rejects a duplicate board code before writing rather than after	4
The consequences of the parameter being named boardCode are worked through,	2

and both [????] attributes are answered correctly	
The views were scaffolded as the brief requires, the four ViewBag properties are honoured so the buttons appear and disappear correctly, and the solution has the structure shown in §10	3

Block C — Comment Coding Standard · 5

Assessed	Marks
Header block on every file your group wrote, member-level comment blocks, and end comments	5

How each item is awarded

Every item is marked **whole**, full marks, half, or nothing. There is no partial credit beyond that.

Award	What it means
Full	Does what was asked, works when run, and is named and spelled as specified.
Half	The right approach, carried out incompletely, or correct, but with a departure that changes what the user sees.
Nothing	Absent, or present but not doing its job.

Two consequences worth understanding:

- **A specified string, name or file location that does not match is worth nothing for that item.** Not “close enough”. The brief gives you every one of them in full. Copy them, do not retype them.
- **Half marks are for incomplete work, not for broken work.** Something that runs but does the wrong thing is not half right.

One mistake is charged once. If a single error breaks the behaviour and is also visible in the code, it costs you the one item it belongs to, not three.

Deductions: the Comment Coding Standard and your student numbers

This is published now, before the assessment opens, and it will be applied. §1.2 of the brief has always said that at least 30 % can be deducted. This is what that means in practice.

There are two separate measurements. Both are made, and the deductions add.

1 — The comment standard

What is found in your source	Result
File header complete on every .cs file your group wrote, method and property blocks present, end comments present	Block C is marked normally (full, half or nothing)
Some of the required commenting is present, but not all, typically the blocks are written and the file header is missing or incomplete	Block C scores 0. No further deduction
None of the three required types is present	Block C scores 0, and a further 10 % of the total is deducted

2 — Student numbers in the file headers

What is found in your source	Result
Two or more student numbers	no deduction
Exactly one student number	5 % of the total is deducted
No student number anywhere	10 % of the total is deducted

The worst case, a submission with no comments and no student numbers, loses Block C plus 20 %, which is the 30 % the brief permits. Nothing worse than that can happen to you on these two grounds.

Why the student numbers matter more than the deduction does

Marks are assigned to people by the student numbers in your source files. There is no other record of who was in your group. In Practical Assessment 1, 29 of 53 submissions carried no student number at all, which left 182 registered students appearing in no submission.

If your submission carries no usable student numbers:

1. The **mark is loaded for the Group Leader only** until the Group Leader supplies the membership by the stated deadline.
2. Supplying the membership afterwards restores the marks to the other members. **It does not reverse the deduction.**
3. A student number that is not on the class list **will not be corrected for you**, however obvious the typo looks. One transposed digit is a different person, and we do not guess a mark onto someone who may not have earned it. The Group Leader confirms the correct number.

The standard is the responsibility of the whole group, not of whoever compiled the submission, so the deduction applies to the group mark like every other item.

Before you submit

This is the whole checklist. It is deliberately short, and it is deliberately not a list of the mistakes we expect you to make. Finding those is part of the assessment.

1. **Build it.** Read the Error List, filtered to Errors. Fix everything there.
 2. **Run the full cycle yourself.** Add a board, see it listed, view its details, change it, confirm the change appears in the list, delete a different one, confirm it is gone.
 3. **Stop the application and start it again.** Everything from step 2 must still be exactly as you left it, and you must still have ten seed boards plus whatever you added, not twenty. **This is the first thing that will be checked, and it is what makes this assessment different from Practical Assessment 1.**
 4. **Delete Database/BoardAppDB.db before you zip your submission.** Keep the empty Database folder. Your application must be able to create and seed the database from nothing, and that is how it will be marked. A submission that only works because your own database file came along with it has not demonstrated anything.
 5. **Compare every specified string, name and folder against the brief,** character by character. Do not trust your memory of them.
 6. **Check the comment header on every file you wrote,** and check that every member's student number is in it.
 7. **Check the folder name.**
-

Academic integrity

By submitting, your group declares the work is its own, in accordance with CUT's Code of Academic Integrity. BoardAppDB is specific to this module and this year. Material found online that solves a different application will not meet these requirements, and code shared between groups is treated as a breach by **all** parties.

Ask if you are unsure whether something is allowed. That conversation is always cheaper than the alternative.